Relatório de Refatoração com Clean Code: RepublicaService

Nome: Maria Eduarda Machado Silva - GU3055469

Disciplina: Engenharia de Software

1) Introdução

Este relatório compara duas versões do `RepublicaService`, a classe principal da API de despesas de república que fiz em Java com Spring. Esse código veio da disciplina de APIs e Microsserviços. Lá, o professor viu o código e pediu para refatorar: dividir o service, usar tipos explícitos no lugar de `var` e deixar só uma instrução por linha. Como esta disciplina pedia um exemplo de código não refatorado e refatorado, aproveitei o mesmo trabalho.

Por enquanto, por falta de tempo, refatorei apenas o RepublicaService. Os controllers, a demonstração e os testes só foram ajustados para chamar os novos serviços. Pretendo aplicar as mesmas etapas nas outras partes do projeto.

2) O que mudou

A mudança mais visível é a estrutura. O `RepublicaService` virou seis serviços, cada um cuidando de um assunto, mais duas classes de apoio e um enum.

	Antes	Depois
Estrutura	Uma classe com tudo	`MembroService`, `DespesaService`, `RecorrenciaService`, `PagamentoService`, `SaldoService`, `FechamentoService`, mais `ContextoRepublica` e `RegrasDespesa`
Nomes	`m`, `r`, `d`, `rc`, `p`; `travar`, `pertence`, `admin`	`morador`, `republica`, `despesa`; `bloquearRepublica`, `exigirMembroDaRepublica`, `exigirAdministrador`
Tipos	`var`	Tipo escrito, como `Republica republica = ...`
Formato	Vários comandos na mesma linha, `if` sem chaves	Uma instrução por linha, `if` sempre com chaves

	Antes	Depois
Métodos	Métodos grandes, fazendo várias coisas	Métodos menores, como `validarSaida` e `criarDespesaDoModelo`
Comentários	Explicavam o que o código fazia	Só os que explicam o motivo de uma regra
Erros	`catch (Exception)`	`catch` de `DateTimeParseException` e `NullPointerException`
Textos fixos	`"DESPESA_PAGA"`, `"PARCELA"` no meio do código	Enum `TipoEvento`

Um exemplo é o método que confere se o usuário é o administrador.

```
private void admin(Republica r,Long usuario) {
if(!r.administradorId.equals(usuario)) throw RegraException.proibido(); }
```

Código antes da refatoração.

```
public void exigirAdministrador(Republica republica, Long usuarioId) {
    if (!republica.administradorId.equals(usuarioId)) {
        throw RegraException.proibido();
    }
}
```

Código depois da refatoração.

Outro exemplo é a edição de uma despesa, que antes ficava em uma linha só.

```
var r=travar(usuario);var d=buscar(usuario,id);editavel(r,usuario,d);return
preencher(d,dto,r);
```

Código antes da refatoração.

```
Republica republica = contexto.bloquearRepublica(usuarioId);
Despesa despesa = buscarDespesa(usuarioId, despesaId);
regrasDespesa.validarAlteracaoDaDespesa(republica, usuarioId, despesa);
regrasDespesa.preencherDespesa(despesa, novaDespesa, republica);
return despesa;
```

Código depois da refatoração.

3) Eixos do Clean Code que usei

- Nomes significativos: troquei as abreviações por nomes que dizem o que cada coisa é e renomeei métodos públicos, como `buscar` para `buscarDespesa` e `pagamentos` para `listarPagamentos`.
- Formatação: uma instrução por linha, chaves em todos os `if` e linhas em branco separando as etapas.
- Funções: quebrei trechos grandes em métodos pequenos, cada um com uma tarefa, e criei variáveis que explicam as condições, como `jaGerada` e `jaFechado`.
- Comentários: deixei só os que explicam o porquê de uma regra. O resto passou a ser explicado pelo nome.
- Tratamento de erros: troquei o `catch` genérico por exceções específicas e mantive as mensagens de erro.
- Classes: cada service ficou com um assunto só, e o que era usado por vários foi para `ContextoRepublica` e `RegrasDespesa`.
- Textos fixos: os tipos de evento do extrato viraram o enum `TipoEvento`, sem mudar o JSON.

4) Dificuldades

Quando fui explicar o código não refatorado, tive dificuldade. Como ele usava `var`, eu não lembrava como cada tipo tinha sido definido e precisava voltar na declaração para entender. Além disso, estava tudo em uma linha só, o que dificultava a leitura.

Na refatoração, a parte mais difícil foi dividir o service, como o professor indicou. Muitos métodos dependiam uns dos outros, então tive que pensar em como os serviços conversam entre si e no que se encaixa melhor em cada um. Por exemplo, para um morador sair da república, o `MembroService` precisa do saldo, que é calculado no `SaldoService`, e o `SaldoService` usa a listagem de despesas do `DespesaService`. Por isso o `calcularSaldos` ficou público. O preenchimento de despesa é usado por despesas e por recorrências, e a trava por república é usada em quase tudo, então essas partes foram para `RegrasDespesa` e `ContextoRepublica`.

Depois da divisão, controllers, demonstração e testes precisaram ser ajustados para os novos serviços e nomes. Os 29 testes automatizados do projeto passaram. Um detalhe técnico: o `ContextoRepublica` não abre transação própria, porque a trava da república só vale enquanto a transação do service que a chamou estiver aberta.

5) Lições aprendidas

A principal lição é que o Clean Code ajuda a entender o código e também os erros. No código antigo, com tudo em uma linha, era difícil ver o que cada trecho fazia. Agora cada método diz o que faz pelo nome, e fica mais fácil achar onde algo deu errado.

Separar por responsabilidade também deixou a conversa entre o controller e as services mais organizada: cada rota chama o serviço do assunto dela.

Vi também que dividir uma classe grande exige pensar antes em quem depende de quem. Sem isso, a divisão cria problemas novos. E, com bons nomes, vários comentários deixaram de ser necessários.

6) O que ainda dá para melhorar

Como só o RepublicaService foi refatorado por enquanto, o resto do projeto (controllers, demonstração, testes e outras classes) ainda mantém o estilo antigo em alguns pontos, como `var` e vários comandos na mesma linha no `RepublicaApiTest`. O `SaldoService` também reúne saldos e relatórios, e as entidades ainda têm atributos públicos. Os testes usam banco em memória, e testes de concorrência em banco real ainda seriam bons. O plano é aplicar as mesmas etapas (nomes melhores, tipos explícitos, uma instrução por linha e divisão por responsabilidade) nas demais partes do projeto.

7) Conferindo com o livro

Conferi a refatoração com o Cap. 9 do livro Engenharia de Software Moderna, do Marco Tulio Valente. O que fiz corresponde aos refactorings de Extração de Método, Extração de Classe, Movimentação de Método, Renomeação e Extração de Variáveis, e resolve os smells de Classe Grande e Métodos Longos. O livro também recomenda reescrever código ruim em vez de comentá-lo, que foi o que aconteceu com os comentários antigos.

Pelo mesmo capítulo, ainda restam três pontos:

- Feature Envy e Classes de Dados: métodos como `envolveMorador` usam só dados de `Pagamento` e poderiam ficar na própria entidade.
- Obsessão por Tipos Primitivos: o mês (`"AAAA-MM"`), a categoria e o tipo de divisão são `String`.
- O nome `ContextoRepublica` é genérico, e o livro cita nomes assim como possível sinal de classe que concentra demais.

8) Conclusão

A refatoração deixou o código mais fácil de ler e de explicar. O mais difícil foi dividir a classe entre os serviços por causa das dependências, e o que mais ajudou foi dar bons nomes e escrever uma instrução por linha.